

-- HP紹介内容 --

研修名

ドメイン駆動設計実践Java編

- 設計から実装へDDDで学ぶWeb API開発 -

キャッチコピー

Javaで学ぶ、ドメイン駆動設計の実践 設計原則の基礎からDDDの4層アーキテクチャ実装まで。ロバストネス分析・Value Object・Entity・Repository・Usecase・Controllerを体系的に習得する、Javaエンジニア必修の研修。

研修概要

■ 本研修について

- 本研修は、ソフトウェア設計の基本思想からドメイン駆動設計（DDD）の完全実装まで、分析・設計・実装を一貫して体験するJava実践プログラムです。
- SOLID原則・GoF23パターン・エンタープライズパターンを設計の土台として習得した上で、ロバストネス分析によるユビキタス言語の定義・概念ドメインモデルの構築・4層アーキテクチャ（ドメイン層・インフラストラクチャ層・アプリケーション層・プレゼンテーション層）の設計と実装を一貫して体験します。
- インフラストラクチャ層はRepositoryパターンとDIPにより抽象化されており、jOOQ・MyBatis・Spring JPAのいずれにも対応可能な設計になっています。既存技術スタックに合わせた柔軟な提供が可能です。

■ 本研修の3つの特徴

本研修では、以下の3つを習得することを目的としています。

1. 「なぜその設計か」を根拠から説明できるようになる SOLID原則・GoF23パターン・DDDパターンをDDDの実装と連結して理解し、設計の意図をチームに言語化して伝えられるエンジニアになる。
2. DDDの設計プロセス全体を自走して実践できるようになる ロバストネス分析から実装まで、DDDの全プロセスを自分の手で体験することで、現場のプロジェクトに即座に適用できる実践力を身につける。
3. どのORMを使っても通用する設計力を習得する Repositoryパターン・DIPの理解により、jOOQ・MyBatis・Spring JPAのいずれのプロジェクトでも、DDDの設計思想を適用できるようになる。

■ 到達目標（受講後に身につくスキル）

本研修を修了すると、以下のスキルが身につきます。

- 設計原則・パターン SOLID原則（単一責任・オープン/クローズド・インターフェイス分離・依存性逆転）をDDDの実装と対応づけて説明・適用できる GoF23パターン・エンタープライズパターンをDDDの各層設計で活用できる 設計の意図をチームに言語化して伝えられる
- 分析・設計プロセス ロバストネス分析によってユビキタス言語を定義し、概念ドメインモデルを構築できる 4層アーキテクチャ（ドメイン層・インフラストラクチャ層・アプリケーション層・プレゼンテーション層）を設計できる 各層の責務と依存関係の方向を正しく設計できる
- DDD実装 Value Object・Entity・Aggregate・Repositoryインターフェイス・Mapperインターフェイスをドメイン層に実装できる DTO・Usecase（Interactor）・Service・Mapper/Assemblerをアプリケーション層に実装できる Controller・Schema・ExceptionHandlerをプレゼンテーション層に実装できる RepositoryImpl・MapperImplをインフラストラクチャ層に実装できる
- ORM対応力 Repositoryパターン・DIPによりjOOQ・MyBatis・Spring JPAのいずれにも対応できる設計を実装できる ORMを差し替えてもドメイン層・アプリケーション層に影響が出ない設計の理由を説明できる

こんな企業にお勧め

以下のような課題をお持ちの企業・チームに特にお勧めします。

- Javaで開発しているが、設計の属人化・技術負債が課題になっている
- DDDを導入したいが、概念の理解と実装のギャップが埋まらないでいる
- jOOQ・MyBatis・Spring JPAなど、チームごとにORMが異なりDDDの統一設計ができていない
- アーキテクト・テックリードを育成し、設計の意図をチーム全体に浸透させたい

日数

3日間

カリキュラム詳細（全3日間）

日程	章・学習テーマ	学習内容・習得スキル
1 日 目	第1章：ソフトウェア設計の基本思想	モジュール化・関心の分離・カプセル化・抽象化・高凝集・疎結合を理解し、DDDの設計思想の土台を築く。
	第2章：設計原則とデザインパターン	SOLID原則・GoF23パターン・エンタープライズパターンをDDDの各層設計と対応づけて習得する。
	第3章：設計プロセス（分析フェーズ）	ロバストネス分析によるユビキタス言語の定義・概念ドメインモデルの構築・モデルコミュニケーションを実践する。
2 日 目	第3章：設計プロセス（設計フェーズ）＋第4章：ドメイン層の設計	4層アーキテクチャへの展開・Value Object・Entity・Aggregate・Repositoryインターフェイス・Mapperインターフェイス・DIPを実装する。

日程	章・学習テーマ	学習内容・習得スキル
3 日 目	第5章：インフラストラクチャ層の設計	MapperImpl・RepositoryImplを実装し、jOOQ・MyBatis・Spring JPAのいずれにも対応できるRepositoryパターンによる抽象化を体得する。
	第6章：アプリケーション層の設計	Service・DTO・Usecase（Interactor）・Mapper/Assemblerを実装し、ビジネスロジックの組み立てと各層連携を実践する。
	第7章：プレゼンテーション層の設計	Controller・Schema・ExceptionHandlerを実装し、REST APIとして4層アーキテクチャを完成させる。Schema→DTO→Entity→Recordの変換レイヤーを実装し、全体を統合する。

※ 各章末に演習を実施します。 ※ 3日目は4層アーキテクチャの総合演習として位置づけます。

前提知識・スキル-> 受講にあたっての前提知識

- プログラミングの基礎知識 Javaを用いた基本的なプログラミング（変数・条件分岐・ループ・クラス・メソッド）ができること。オブジェクト指向プログラミング（クラス・インターフェイス・継承・ポリモーフィズム）の基本を理解していること。例外処理（try-catch・カスタム例外）の基本を理解していること。
- Web・APIの基礎知識 HTTPリクエスト・レスポンスの基本概念（GET・POST・PUT・DELETE・ステータスコード）を理解していること。REST APIの基本概念（エンドポイント・JSONデータ）を理解していること。
- データベースの基礎知識 リレーショナルデータベース（テーブル・主キー・外部キー）の基本概念を理解していること。SQLによる基本的なCRUD操作（SELECT・INSERT・UPDATE・DELETE）ができること。
- あると望ましい知識 推奨 Spring Boot（DIコンテナ・Bean定義・アノテーション）の基礎知識。jOOQ・MyBatis・Spring JPAのいずれかの使用経験（操作経験は不要）。GitHubを使ったりリポジトリのクローン・プッシュなど基本的な操作ができること。

必要な受講環境

区分	環境 / ツール	役割・用途
OS	Windows 11 / macOS	研修用PCの基本環境
IDE	IntelliJ IDEA（Community版以上）	Javaコード編集・デバッグ・ビルド
JDK	Java 21 LTS	Javaアプリケーションの実行環境
フレームワーク	Spring Boot 4.x	REST APIアプリケーションの構築基盤
ビルドツール	Gradle	依存関係管理・ビルド・実行

区分	環境 / ツール	役割・用途
OR Mapper	jOOQ / MyBatis / Spring JPA	データベースアクセス実装（顧客環境に合わせて選択）
RDBMS	PostgreSQL	演習アプリのデータベース
バージョン管理	GitHub（アカウント）	演習リポジトリの取得・管理

※ ORMはjOOQ・MyBatis・Spring JPAのいずれにも対応しています。顧客環境に合わせて選択できます。